

One Person Lab

OPL Framework 白皮书

为长期运行的 AI 专业工作提供可组合、可验证、可演进的底座

优秀的智能体底座不应把能力焊死在一个运行器里，而应让每项能力都能独立理解、替换、验证和组合，同时始终保留清楚的事实与责任边界。

Cordis Host / Capability Domains / App Client Cordis / Governed Evolution

2026-08-15

Public whitepaper

目录

1	从“能运行”到“值得长期托付”	2
2	为什么选择 Cordis	2
3	四层生态中的可组合底座	2
4	四种身份，分别回答四个问题	3
5	自由组合，不把复杂度交给用户	4
6	Framework Host 与 App Client：同一架构，两种职责	4
7	两套 GUI，一份产品承诺	4
8	自进化：改进模块，而不是重写整个智能体	4
9	可组合的边界，也是可信的边界	5
10	一次升级如何发生	5
11	为什么这套设计值得相信	5
12	结语	5

发布日期：2026-08-15

最近修订：2026-08-15

适用对象：希望理解 OPL Framework 为什么这样设计、它如何支撑可靠的 AI 专业工作，以及这套架构为什么能够长期演进的用户、合作者和技术决策者。

核心判断：优秀的智能体底座不应把能力焊死在一个运行器里，而应让每项能力都能独立理解、替换、验证和组合，同时始终保留清楚的事实与责任边界。

1 从“能运行”到“值得长期托付”

让一个智能体完成一次任务并不困难。真正困难的是让一组不断变化的模型、工具、专业能力和用户界面，在数周甚至数月的工作中仍然保持可理解、可恢复和可追责。

复杂知识工作会不断遇到变化：更好的模型出现了，新的资料源需要接入，执行方式需要替换，评估方法需要升级，界面也会迭代。如果这些能力都依赖同一套隐式启动顺序、全局状态和手工连接，任何一次升级都可能牵动整条运行链。系统越有能力，开发、测试和运维反而越困难。

OPL Framework 要解决的不是“如何再造一个更大的智能体”，而是“如何让专业智能体的每一部分都能持续变好，而整个系统仍然可信”。

因此，Framework 把运行底座理解为一个可组合的能力系统。规则、发现、工作空间、阶段、执行、证据、呈现、评估和连接不再是一个黑箱中的内部步骤，而是由 capability domain、authority、Package 和 Cordis contribution 共同表达的责任面。它们共同服务一条专业工作线，却不互相吞并事实和判断权。

2 为什么选择 Cordis

OPL Framework 选择 Cordis，不是为了追随一种技术形式，而是因为它提供了一个与长期目标一致的基本语法：能力通过明确的依赖进入组合，在清楚的生命周期中启动和退出，并能被检查、替换和重新组合。

这带来五个直接结果。

第一，**改变一个能力，不必先理解整个系统**。模块只需要理解自己的责任和明确依赖，升级执行器、目录或观测能力时，改动可以停留在合理边界内。

第二，**测试从整机验收变成模块证据**。每个模块可以独立装载、注入受控环境、面对缺失依赖并证明退出后没有遗留影响；完整组合再验证模块之间是否协作正确。

第三，**故障可以被解释**。系统能够回答实际装载了什么、哪项能力由谁提供、哪个版本参与了本次工作，以及不兼容发生在哪里。

第四，**升级和回退有了稳定对象**。一次工作可以绑定到明确的组合快照。新模块可以先在受控组合中验证，出现问题时也能回到上一组已知配置，而不必依赖难以复现的全局状态。

第五，**自进化有了真实的操作单位**。候选生成、独立评估、受控替换和效果比较可以围绕一个模块进行，不必每次修改整个智能体运行时。

Cordis 提供的是组合秩序，而不是专业裁决。它不会替代领域 Agent 的判断，不会接管成果权威，也不会把“成功装载”解释为“专业质量合格”。这种克制正是 OPL 采用它的原因之一。

3 四层生态中的可组合底座

用户面对的是 OPL Base、OPL App、OPL Packages 和按需使用的 OPL Cloud。Framework 是 Base 的核心实现和唯一 Cordis Host；Packages 是可独立安装与发布的专业能力；App 是本地工作台；Cloud 负责在线治理与托管。Cordis 是 Base 内部的组合机制，不是第五个产品层。

Framework 当前可以装载多类 Cordis contribution。它们说明运行时有哪些可替换责任，但不等于固定数量的外部产品、Package、仓库或品牌目录；真实数量由 caller、生命周期和替换价值决定。

领域	当前 contribution	对读者的核心承诺
规则与认知	OPL Charter	让共同规则与权限边界保持明确，能力升级不改变系统承诺
规则与认知	OPL Atlas	让可用 Agent、Package 与能力可以被发现和理解，而不是藏在固定清单里
规则与认知	OPL Workspace	让每项工作始终回到正确的材料、文件与工作位置
能力与阶段	OPL Stage Pack	把阶段所需的专业能力组织成可以安装、版本化和复用的单元
能力与阶段	OPL Package Host	让不同 Package 在统一兼容边界内进入 Framework，而不复制一套运行底座
能力与阶段	OPL Stagecraft	为一次专业阶段准备清楚的目标、上下文和边界，同时保留 AI 的开放判断
执行与恢复	OPL Runway Bridge	把阶段意图可靠地交给实际运行环境，并隔离底层执行差异
执行与恢复	OPL Runway Executor	让一次尝试真正启动、持续运行、恢复并形成可交接结果
证据与操作	OPL Ledger	保存可追溯的事件与证据引用，让进展、故障和决定可以复查
证据与操作	OPL Console	把复杂运行事实转化为用户和操作人员能够采取行动的清楚视图
演进与评估	OPL Foundry Provider	让候选能力以明确来源和身份进入演进过程
演进与评估	OPL Foundry Evaluation	用独立证据比较候选方案，避免“能运行”被误当成“更优秀”
生态与交付	OPL Connect Discovery	让外部能力和贡献按统一方式被发现，而不是继续扩张中央注册表
生态与交付	OPL Connect Release	让经过确认的能力进入可追踪的交付过程，并保留版本与来源关系

这些 contribution 是当前运行组合的可检查入口。对外品牌和源码导航把相近责任组织成 capability domains；一个认知域可以贡献多个 contribution，也可以只保留 authority surface。数量只跟随真实责任和 caller，不成为架构目标。

4 四种身份，分别回答四个问题

降低认知成本，不是强迫所有分类一对一，而是让每种分类只回答一个问题：

身份	回答的问题
品牌模块 / capability domain authority owner	人怎样理解跨 Framework、App 与 Cloud 的稳定责任？ 谁可以决定事实、质量、权限、状态或发布？
OPL Package	哪些能力可以独立安装、发布、升级和分发？
Cordis plugin contribution	哪段进程内能力需要独立挂载、注入、隔离和卸载？

因此，一个 Package 可以贡献多个 plugin，也可以只提供 Skill、Tool、Workflow 或静态能力；一个品牌模块可以跨多个 Package 与 plugin；纯 authority domain 也可以没有常驻 plugin。composition snapshot 只冻结一次运行选择，不能替代 Package currentness 或 owner authority。

独立 Package、独立版本线和独立仓库优先落到真正具有独立发布节奏和替换价值的 Runway Executor、Foundry Evaluation、Connect Discovery、Package Host 与 Cordis ABI 等单元。Framework 先在同一 monorepo 建立独立 workspace、API/version/source identity 和组合快照，以最小管理成本 证明边界；只有 Package owner 的真实发

布与 channel readback 完成后，才称为独立发布。其他稳定 贡献可以继续共处一仓。是否继续拆分，由真实 caller、故障隔离和发布证据决定，而不是为了保持 目录或数字对称。

5 自由组合，不把复杂度交给用户

可组合不等于让用户面对任意插件开关和难以验证的组合矩阵。OPL Framework 在内部保留贡献组合自由，在外部提供少量经过验证的组合形态。

无界面的基础运行、完整桌面工作台、研究工作、基金工作、视觉交付和 Foundry 开发，需要的能力组合并不相同。Framework 用受控 Profile 表达这些差异：每个 Profile 说明自己需要哪些受控 contribution，冻结一次实际组合，并接受完整验证。

用户选择的是“我要完成什么工作”，而不是“我要装载哪些插件”。capability-domain 认知降低维护成本，Profile 降低用户认知成本。两者结合，系统内部可以快速演进，产品表面仍然保持稳定、清楚和可预测。

6 Framework Host 与 App Client：同一架构，两种职责

OPL Framework 是唯一的 Cordis Host。它依据受控 profile 决定哪些 runtime contribution 能够进入一个组合，管理它们的生命周期，并投影出经过 App 准入的能力和状态。

One Person Lab App 不是另一套平行 Host。它是 App 产品 authority 加上一组面向用户体验的受控 GUI contributions：窗口、导航、任务视图、文件工作面、进度呈现和操作入口可以由 Host 投影的 graph 派生到 Client Cordis 中，但它们不能自行发现特权插件，也不能接管 Package、运行事实或专业判断的权威。

因此，App 侧可以拥有由 Host graph 派生的 Client Cordis，用来组织界面贡献和客户端生命周期；Framework Host 仍然是组合权威，App 仍然是产品准入、页面结构和发布 authority。Host 与 Client 使用一致的插件思想，却承担不同责任：Host 决定能力真实存在并允许被消费，Client 决定这些能力怎样成为连贯的用户体验。

这一区分让 GUI 插件化不会演变成第二个 Framework，也让后端模块化不会把界面锁死在某一种实现上。

7 两套 GUI，一份产品承诺

One Person Lab App 当前同时推进两条 GUI 路线：基于 AionUI 的主线版本，以及基于 DeepSeek Harness GUI 的候选版本。它们可以拥有不同的渲染技术、交互实验和演进速度，但不能拥有两套产品事实。

两条路线都遵循同一顶层设计：Framework Host 投影 allowlisted graph，App Client Cordis 组织 GUI 贡献，产品合同定义用户应当看见的任务、Package、状态、动作和责任边界。AionUI 主线与 DSH-derived candidate 共享这份 ABI、typed slots/actions、RPC/events 和 product profile；只有 renderer、carrier 和视觉实现可以不同。

这使双 GUI 不再意味着双倍架构成本。AionUI 主线可以继续承载稳定产品体验，DSH-derived 候选可以更快学习新的交互与插件机制；任何成熟能力都必须通过共同产品合同进入，而不是在某个 Shell 内形成新的事实来源。

8 自进化：改进模块，而不是重写整个智能体

Harness 的长期价值，不只是把工具接给模型，而是让系统能够观察自身、提出改进并在证据约束下采用更好的实现。

当执行、发现、观测、评估和连接都是独立 contribution 或 Package 时，Foundry 可以围绕一个清楚对象工作：冻结当前版本，生成候选，执行针对性测试，在真实 Profile 中比较效果，再由正确的责任方决定是否激活。成功的改进只替换相应贡献；失败的候选不会污染整个运行底座。

这比“让 Agent 修改自己的全部代码”更可控，也比永远依赖人工整体发布更有演进效率。模块身份提供实验单位，组合快照提供复现边界，Ledger 提供证据，Foundry Evaluation 提供独立比较，最终采用权仍然属于明确的 owner。

自进化因此不是一个神秘的自动循环，而是一条专业的改进链：提出候选、获得证据、接受评估、受控激活、保留回退。

9 可组合的边界，也是可信的边界

OPL Framework 不把所有责任都吸收到插件系统中。可组合层负责运行能力之间的协作，但几类事实必须留在真正的拥有者手中。

领域 Agent 决定专业成果是否合格；Workspace 和文件系统保存实际材料与产物；持久运行环境保存长期执行历史；Ledger 保存证据与事件；App 持有桌面产品和发布事实；Package owner 持有自己的来源、版本与能力声明。Framework 连接这些事实，不复制它们，也不因为某个模块装载成功就替它们宣布就绪。

这种边界看似克制，实际上让系统更容易扩展。新的 provider、connector、observer 或 executor 可以进入组合，却不能悄悄获得不属于自己的权威。创新发生在局部，信任保持在全局。

10 一次升级如何发生

假设 Runway Executor 出现了一个更擅长长期任务恢复的候选版本。

在传统整体式运行器中，这次升级可能同时触碰启动逻辑、状态页、工具注册、任务恢复和发布脚本。团队需要重新验证整个系统，而且很难判断改进究竟来自哪里。

在 OPL Framework 中，候选版本拥有自己的模块身份。它先在受控 Profile 中替换现有 Runway Executor，沿用相同的 Stagecraft 上下文、Workspace 位置和 Ledger 证据边界。Foundry Evaluation 比较恢复成功率、证据完整性和用户可见结果。只有证据支持且 owner 接受后，新组合才成为后续工作的选择。

升级前后的工作都能回答：使用了哪个模块版本，组合中还有哪些能力，评估依据是什么，出现问题时回到哪里。一次局部改进因此不会变成一次全系统赌博。

11 为什么这套设计值得相信

OPL Framework 的专业性不来自模块数量，而来自一组持续兑现的工程承诺。

- **概念与实现一致。** 品牌模块是 capability-domain 认知地图；authority、Package、插件和版本身份分别由各自真实边界表达。
- **组合可以被看见。** 每次运行都能解释实际装载的能力及其来源，而不是依赖隐式全局状态。
- **升级可以被比较。** 候选模块先获得独立证据，再进入受控组合，不以“已经接入”代替“已经更好”。
- **故障可以被隔离。** 模块生命周期和责任边界让问题停留在合理范围，并保留清楚的回退对象。
- **产品可以换壳而不换事实。** AionUI 主线与 DSH-derived 候选共享同一 Host 投影和 App 产品合同。
- **权威不会随便利漂移。** Framework 负责组合与运行，领域质量、文件、发布和最终决定仍归正确的 owner。

这套设计让 OPL 可以同时获得两种通常难以兼得的能力：内部持续变化，外部长期稳定。

12 结语

OPL Framework 的目标，不是成为一个越来越庞大的中央运行器，而是成为一套越来越清楚的专业能力系统。

四层生态让用户只需理解 Base、App、Packages 和可选 Cloud；正交的认知域、authority、Package 与 plugin contribution 让开发者能准确定位责任；Cordis 让运行贡献可以组合、检查和替换；受控 Profile 让内部自由不会成为用户负担；Framework Host 与 App Client Cordis 让运行和界面共享同一种设计语言；Foundry 则让每项能力可以在证据约束下持续改进。

最终，用户不需要感知插件系统本身。他们感知到的是更稳定的工作、更清楚的进展、更容易恢复的任务，以及一个能够不断升级却不丢失责任与事实的 AI 专业平台。

了解更多：

- [One Person Lab 家族白皮书](#)
- [One Person Lab](#)
- [One Person Lab App](#)
- [OPL Cloud](#)